

Introduction to IT

Lecture 1

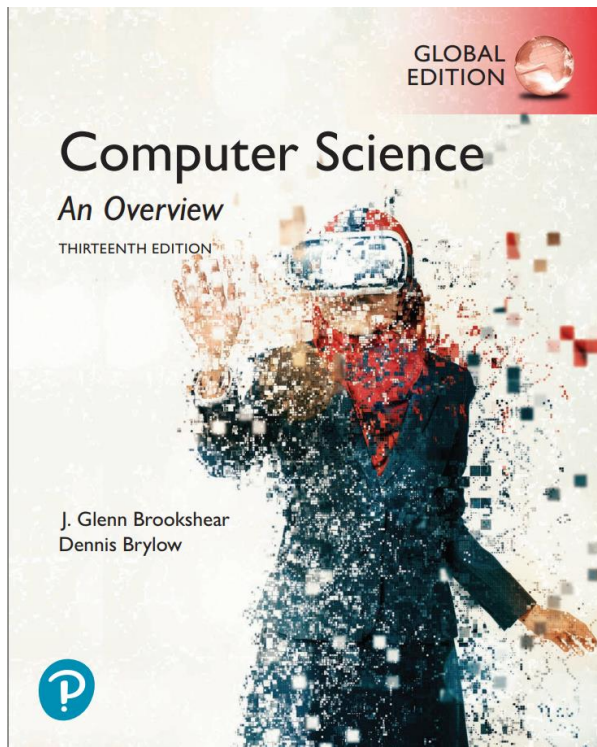
Course Info

- Lecturer: Dr. Sarvar Abdullaev
- Email: s.abdullaev@inha.uz
- Office hours:
 - Thursday 12:00 – 13:00 in Room B411 (or A203)
 - Friday 15:00 – 16:00 in Room B411 (or B210)
- Telegram Channel:
 - **https://t.me/intro2it_2022**
- Course format:
 - 2 lectures every week
 - 1 homework assignment every week

Textbooks

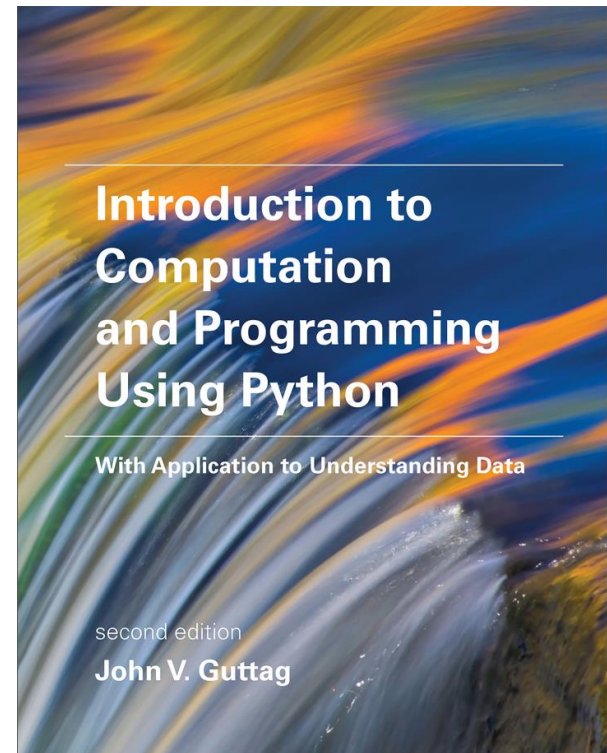
- **For Lectures:**

- Brookshear, J. Glenn, Brylow, D., *Computer science: An Overview*, 13th Ed, Pearson, 2020.



- **For Homeworks:**

- Guttag, John V. *Introduction to Computation and Programming using Python*. MIT Press, 2013.



Evaluation Criteria

- 30% Mid-term exam (8th week)
- 30% Final exam (15th week)
- 15% Homeworks
 - All lab exercises are assigned in **eClass** (i.e. <http://eclass.inha.ac.kr>)
 - All completed lab exercises must be submitted to **eClass** before the deadline (usually 1 week after assignment)
 - Homework exercises are **NOT GRADED**, but **COUNTED**. If you complete all tasks in your homework, it will be checked as completed.
 - **No excuses for not knowing how to use eClass or relevant tools**
- 15% Quiz (at least 3-4 pop quizzes during the semester)
 - Quiz date is not announced beforehand
 - 15-minute long in-class written exam before the lecture
 - **Only one attempt is allowed. If you miss it without a valid excuse, then it's gone.**
- 10% Attendance
 - Attendance checks are done selectively
- **All marks are published in eClass System!**

Getting 0 for your assignment

You will **get 0** for your assignment if **ANY** of below is **TRUE**:

- You cheated your assignment
 - Cheating is an act of unauthorized copying from other sources, OR an attempt to do so.
 - First incident of cheating is penalized by 0 mark for given assignment only. Second incident results in **FAIL** for the whole course.
- You submitted your assignment **AFTER** the deadline.
 - For example, if deadline is set to 12:00 AM (midnight), and you submit your work to the system at 12:01 or later, it is considered as late submission and penalized by 0 mark.
- Your submitted corrupt files or unreadable work
 - It is your responsibility to ensure that your file is not corrupt and can be viewed by Word, PDF reader or any appropriate software.
 - Make sure that your handwriting is neat and readable, and cannot be interpreted in multiple ways.

Getting **FAIL** for the semester

You will **FAIL** the course if **ANY** of below is **TRUE**:

- Your marks for **both** MID and FINAL exams are less than 30%
- Your total mark from all assignments is less than 30%
- You have been caught cheating for the **second time**.
- You have missed more than 4 weeks of your studies without a valid excuse.

Introduction to Computer Science

... and Programming using Python

What is a computer?

Labrador



Abacus

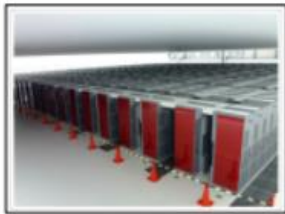


An Array of Computers

Super Computers



Cray-1 (1979): 80 MHz, 8 MB RAM, 2.4 GB disks



K computer (2011)[†]: Fujitsu, 705024 2GHz Sparc64 processors

Workstation/Desktop



Sun UltraSPARC IIIi (2005): 650 MHz, 2 GB RAM, 80 GB HD



Intel Pentium 4 (2005): 3.4 GHz, 4 GB RAM, 300 GB HD

Notebook



Intel Core i5 (2011): 2.4 GHz, 8 GB RAM, 500 GB HD

Smart Phone/ Tablet



ARM-11 (2010): 436 MHz, 128 MB RAM, 16 GB Flash



ARM Cortex A8 (2010): 1 GHz, 256 MB RAM, 64 GB Flash

What is Computer Science?

- 'Computer' is a machine for manipulating data according to *a list of instructions* known as a program.

Data Manipulation = Data Input + Data Processing + Data Output

- Computer Science:
 - A discipline that seeks to build a scientific foundation for *computer design* and *information processing using computers*.
 - *How to design a computer?*
 - *What to do with a computer?*

Computing Systems

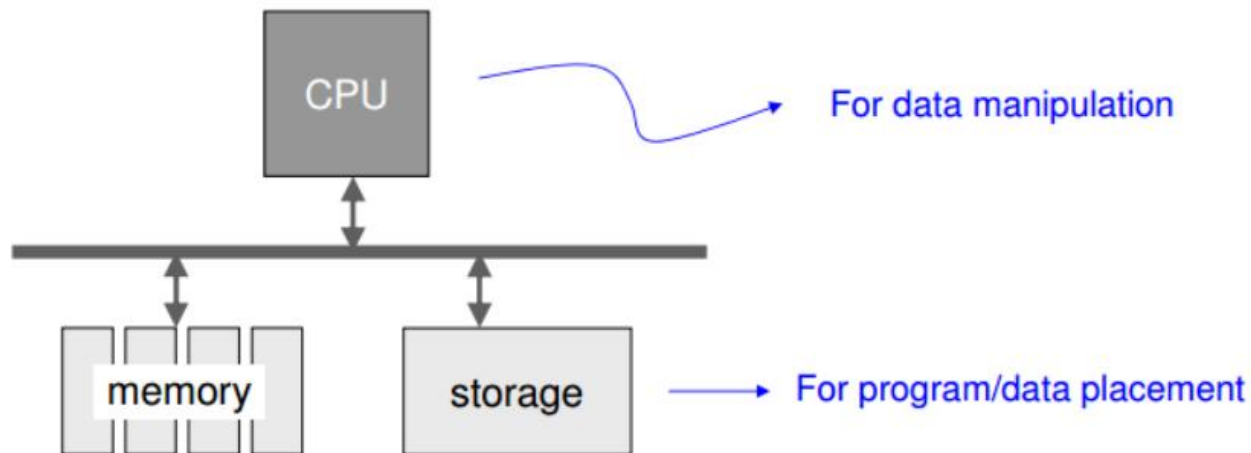
- ❑ A computer alone does not do anything!
- ❑ To complete some useful tasks, we need a “Computing System”:

A computing system is a precisely coordinated operation of **hardware**, **software**, and **input data** that produces proper **output data**

- ❑ Software: a collection of programs
- ❑ Program: an implementation of an algorithm and data structure using a particular language
 - Algorithm → a set of instructions that define how to carry out a task (by manipulating data)
 - Data structure → how the data is organized in memory and storage
- ❑ Hardware: CPU + memory + storage + I/O devices

Key Components of Computers

- ❑ Central Processing Unit (CPU)
- ❑ Volatile Memory (RAM)
- ❑ Non-volatile Storage (Harddisk, Flash)



Examples of 'Programs'

□ Task: compute the sum of one, two, and three.

- Program 1 (written in the English language):

```
Compute the sum of one, two, and three.
```

- Program 2 (written in the C language):

```
for (idx = 1, sum = 0; idx <= 3; idx++) sum += idx;
```

- Program 3 (written in the x86 assembly language):

```
mov ax, 1  
mov bx, 2  
add ax, bx  
inc bx  
add ax, bx
```

Algorithms

Al-Khwarizmi = Algorithm

- **Muhammad ibn Musa al-Khwarizmi**
 - Born in 780, Khiva, Uzbekistan
 - Died in 850, Baghdad, Iraq
 - Prominent Works:
 - Al-Jabr wal Muqabala
 - Zij al-Sindhind
- **Contributions:**
 - Introduced decimal system to Western world
 - Invented Algebra and Algorithm to express algebraic manipulations
 - 116 mathematical tables describing the movements of the Sun, Moon, and 5 other planets known at the time.



Roman way of thinking

- Roman numeric system:
 - I = 1, V= 5, X=10, L=50, C=100, D=500, M=1000
 - MCMLIV=1954, MMMCCMXLIX=3849
- Poor Romans had to deal with:
 - What is MMCCXCIII+CDXLIII=?
 - What is the area of this carpet?
 - OK, he claims the area of this square carpet is 2, then what is its width(height)?
 - Which portion of this land is mine?



Indian way of thinking

- Radix r representation:
 - Any number can be represented with r list of symbols (digits)
 - Increment the digit until:
 - It reaches the desired number
 - OR it reaches the end of the list, then:
 - Increment the adjacent digit on the left
 - Reset the digit to the beginning of the list
- Indian numeric system:
 - 0,1,2,...,9 digits
 - Radix 10 representation



Eureka! I know what to do!



- What if I could solve Romans' headache with Indian numerical system?
- What if I could express my solution universally for every instance of given problem?
- For example:
 - $\text{MMCCXCIII} + \text{CDXLII} \Leftrightarrow 2293 + 442$
 - Want solution? try my brand-new arithmetic algorithm for adding
 - What is the area of this carpet?
 - Jeez, let width be A , height be B , it is $S = A * B$. See, I don't even need to know the actual measurements. Follow my algorithm expressed via algebra.
 - OK, he claims the area of this square carpet is 2, then what is its width(height)?
 - So $S = 2$, also $S = A * A = A^2 \Rightarrow A = \sqrt{S}$, You need square-root, dude! Follow **Babylonian algorithm** for that.
 - Which portion of this land is mine?
 - Oh, you need fractions. Use same Radix 10 principle for fractions.

The Euclidean algorithm

Description: This algorithm assumes that its input consists of two positive integers and proceeds to compute the greatest common divisor of these two values.

Procedure:

Step 1. Assign M and N the value of the larger and smaller of the two input values, respectively.

Step 2. Divide M by N, and call the remainder R.

Step 3. If R is not 0, then assign M the value of N, assign N the value of R, and return to step 2; otherwise, the greatest common divisor is the value currently assigned to N.

The Babylonian Algorithm

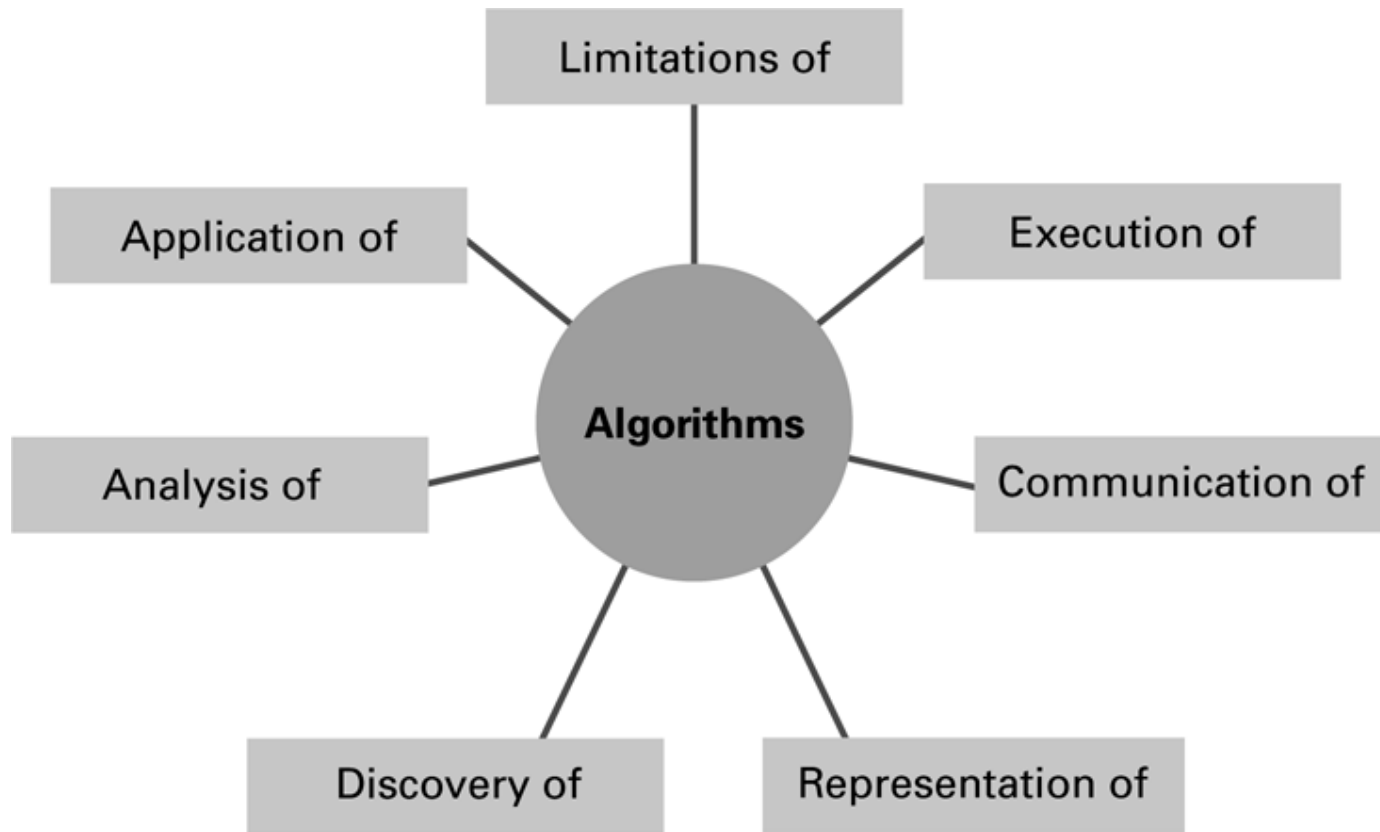
- **Description:** Finds the square-root of any positive number S
- **Requires:** Initial guess $X_0 \approx \sqrt{S}$
- **Method:**
 - Repeat for better precision:

$$X_{t+1} = \frac{1}{2} \left(X_t + \frac{S}{X_t} \right)$$

Key Topics about Algorithms

- Which problems can be solved by algorithms?
- How can algorithm discovery be made easier?
- How can techniques of representing and communicating algorithms be improved?
- How can characteristics of different algorithms be analyzed and compared?
- How can algorithms be used to manipulate information?
- How can algorithms be applied to produce intelligent behavior?
- How does the application of algorithms affect society?

Key Topics about Algorithms



Algorithms are not always a Panacea

- **Gödel's Incompleteness Theorem:** Some problems cannot be solved by algorithms.
- Gödel proved that if you start a “*math*” with set of axioms like:
 - Parallel lines never cross
 - If $A=B$, and $B=C$, then $A=C$, etc.
- Then there still will be laws that are true, but you will never be able to prove them without adding new axioms to your “*math*”.
- You can never prove the consistency of your math with itself.
- Thus, there is no algorithm for proving the consistency of other algorithms.

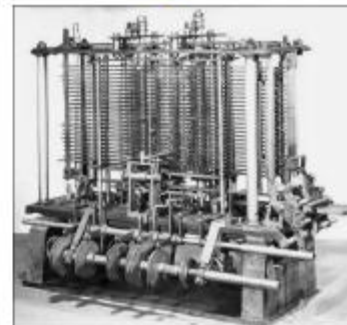
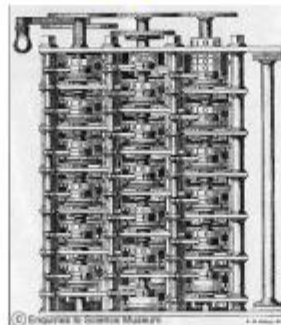
Computing Machines

Origins of Computing Machines

- Early computing devices
 - Abacus: positions of beads represent numbers

- Gear-based machines (1600s-1800s):

- Pascal's adding machine (1642)
 - Leibniz's calculator (1671)
 - Babbage's Difference Engine (1832) and Analytical Engine (1833)



Programmable vs Hardwired

- ❑ Early computers based on gears are not “easily” programmable – the hardware must be modified in order to perform a different task
 - The concept of “programmable” computers was also invented in 1800’s (i.e. Analytical Engine)
 - “Programmable computers” is also refer to as “general-purpose computers”
- ❑ Today, hardwired (non-programmable) computing systems are as popular (if not more popular) as programmable computing systems

Babbage's Difference Engine

- Let's say we need to compute $f(x) = x^2$
- We know from Calculus: $f'(x) = 2x, f''(x) = 2$
- Let's say we know $f(0) = 0, f(1) = 1, f(2) = 4$
- We can use this data to compute $f(3)$ or $f(4)$ or any number mechanically.

x	x^2	First difference	Second difference
0	0		
1	1	1	2
2	4	3	2
3	9	5	2
4	16	7	2
5			

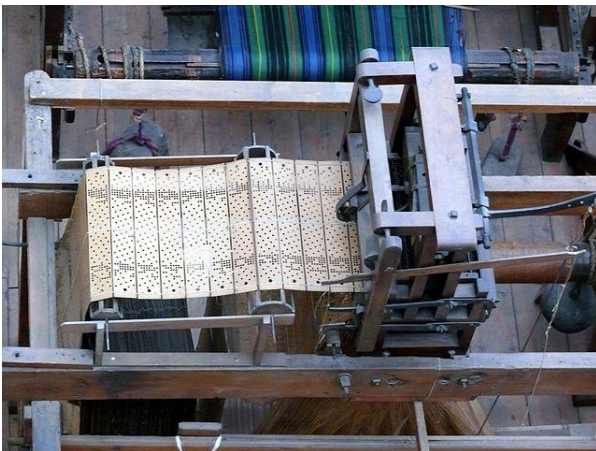
Diagram illustrating the calculation of $f(4) = 16$ using the difference engine. The table shows the values of x , x^2 , the first difference, and the second difference. Arrows indicate the flow of data: from x^2 to the first difference, and from the first difference to the second difference. The value 16 is highlighted in blue, and the value 7 is also highlighted in blue.



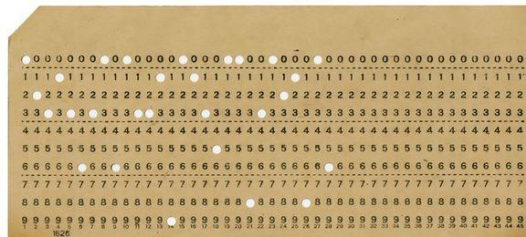
Charles Babbage

Early Programs

- ❑ Early data storage: punched cards
 - First used in Jacquard's loom (1801) to store patterns for weaving cloth
 - Also used to store programs in Babbage's Analytical Engine
 - Ada Byron, who wrote a "program" for this machine, was recognized as the first programmer in the world
 - In 1887, Herman Hollerith invented punch cards to store programs for computers, which were popular through the 1970's and are still used today for data input



Jacquard's Loom



Hollerith's Punch Cards



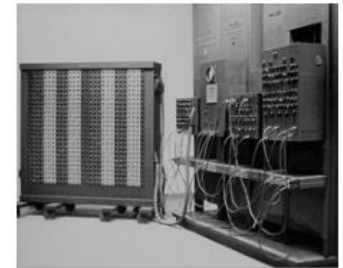
Ada Byron-Lovelace

Early Computers

- Based on mechanical relays
 - 1940: Stibitz at Bell Laboratories
 - 1944: Mark I: Howard Aiken and IBM at Harvard
- Based on vacuum tubes
 - 1937-1941: Atanasoff-Berry at Iowa State
 - 1940s: Colossus: secret German code-breaker
 - 1940s: ENIAC: Mauchly & Eckert at U. of Penn.



→ Mark I

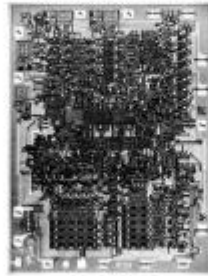


← ENIAC



Microprocessors: Age of Transistors

- ❑ The first microprocessor is Intel 4004 (1971)
- ❑ The first computer based on microprocessor is Intel SIM4-01



Intel 4004



Intel SIM4-01

Personal Computers

- First microprocessor used in “PC” is 8008 (1972). The “PC” was named as Micral (1973)
- The first popular PC was the Apple II (1977) designed by hobbyist Steve Wozniak. CPU was MOS 6502 (1 MHz).
- IBM PC was available in 1981. The CPU was the Intel 8088 (4.77 MHz).



Steve Wozniak



Personal Computers

- First popular PC Operating System was CP/M – Control Program for Microcomputers, written by Gary Kildall in 1973 for the Intel 8080 processor.
- When IBM decided to make IBM PC, they first asked Gary Kildall to port the CP/M to the 8088 processor.
 - However, he refused
 - Later, IBM turned to Microsoft for an OS.
 - Microsoft bought QDOS from Seattle Computers and re-labelled it as MS-DOS for IBM
- In fact, QDOS was hackers' port of CP/M to x86 architecture



Open Source Movement

- UNIX operating system has been developed by Ken Thompson and Dennis Ritchie in 1969, and its source code was made open to public with usage restrictions.
- In 1983, Richard Stallman from MIT launched the GNU project to create a free Unix-like OS. In 1985, FSF is established.
- Linux (by Linus Torvalds) and the GCC toolchain (by Stallman) are both from the GNU project.
- Nowadays, most of the private companies like Google, Microsoft, Facebook, etc join the open source movement, and make some of their work open to public.



Into the Millennia

- World Wide Web:
 - World Wide Web (Sir Timothy John Berners-Lee created his first web page on Aug 6, 1991 at CERN)
 - <http://info.cern.ch/hypertext/WWW/TheProject.html> is Internet's first web page.
 - Web pages could use **hyperlinks** to reference each other, making up so called “web”.
 - First web browsers like Mosaic (1993) and Netscape Navigator (1994) have been created to render such web pages into readable and accessible view.
- Search Engines:
 - 2 PhD students from Stanford Larry Page and Sergey Brin founded Google on Sept 4, 1998.
 - They developed PageRank algorithm to index web pages based on their importance.

Into the Millennia

World Wide Web

The WorldWideWeb (W3) is a wide-area [hypermedia](#) information retrieval initiative aiming to give universal access to a large universe of documents.

Everything there is online about W3 is linked directly or indirectly to this document, including an [executive summary](#) of the project, [Mailing lists](#), [Policy](#), November's [W3 news](#), [Frequently Asked Questions](#).

[What's out there?](#)

Pointers to the world's online information, [subjects](#), [W3 servers](#), etc.

[Help](#)

on the browser you are using

[Software Products](#)

A list of W3 project components and their current state. (e.g. [Line Mode](#), [X11 Viola](#), [NeXTStep](#), [Servers](#), [Tools](#), [Mail robot](#), [Library](#))

[Technical](#)

Details of protocols, formats, program internals etc

[Bibliography](#)

Paper documentation on W3 and references.

[People](#)

A list of some people involved in the project.

[History](#)

A summary of the history of the project.

[How can I help ?](#)

If you would like to support the web..

[Getting code](#)

Getting the code by [anonymous FTP](#), etc.



Search the web using Google!		
Google Search		I'm feeling lucky
Special Searches Stanford Search Linux Search	Help! About Google! Company Info Google! Logos	Get Google! updates monthly: your e-mail Subscribe Archive

Copyright ©1998 Google Inc.

Suggested Activity

- **To Register:** Recommended MOOC
 - MIT's Intro to Computer Science and Programming using Python
 - <https://learning.edx.org/course/course-v1:MITx+6.00.1x+2T2016>
- **To Watch:** BBC Documentaries
 - Ada Lovelace and early computers
 - <https://www.youtube.com/watch?v=QgUVrzkQgds>
 - The Secret Rules of Modern Living Algorithms
 - <https://www.youtube.com/watch?v=kiFfp-HAu64>
- **To Read:** Know thyself!
 - Teach Yourself Computer Science
 - <https://teachyourselfcs.com/>
 - Google's guide for aspiring computer scientists and programmers,
 - <https://www.google.com/about/careers/students/guide-to-technical-development.html>
 - What Computer Scientists Should Know?
 - <http://matt.might.net/articles/what-cs-majors-should-know>

What are we going to study next?

- Data Storage
- Data Manipulation
- Operating Systems
- Networking and Internet
- Algorithms
- Programming Languages
- Software Engineering
- Data Abstractions
- Databases
- Artificial Intelligence ??? (if we will have time)